Experiment Plan

Collecting runtime-q episode data across GPU architectures and MoE expert models

Item	Value
Date	15 October 2026
Repository	github.com/ankitsblade/FreeToken-Reproduce (branch main)
Study package	experiments/runtime_q (original contributor: Saketh), plans in configs/runtime_q
Status	Plan. Collection on new architectures has not started; no new results are reported here.

In one paragraph. On one machine (RTX 4070, gpt-oss-20b) a reuse-aware rule for choosing q - the number of cache-missing experts fetched to the GPU per MoE layer - was 1.52x faster than FreeToken's bandwidth rule q* in end-to-end decode. That result is single-machine and single-model. This plan collects the same episode data on other GPU architectures (RTX 4090, RTX 5090 and others as they become available) and on other MoE expert models, using a GitHub repository that sets up and runs the identical collection on any supported Linux NVIDIA machine. The goal is to find out whether the q-selection rule, and the runtime variables it depends on, transfer.

1. Why this experiment

FreeToken's hybrid decode splits each layer's m cache-missing experts: q are copied to the GPU over PCIe, m - q run on the CPU, concurrently. FreeToken chooses q* = round(m x B_P / B_H), where B_P is the PCIe expert-transfer bandwidth and B_H the CPU expert-processing bandwidth.

The completed single-machine study found:

- q* is correct for one decode step, but on a PCIe-slow host it rounds to 0, so the GPU expert cache is never filled and every expert is recomputed on the CPU.
- A rule that adds the expected reuse of a fetched expert - fetch while p x T_res covers the exposed fetch cost, with rho = B_P / B_H, p = top_k / E and T_res = the LRU victim's age - was 1.52x faster than q* across three cache sizes and 1.70x on an unseen prompt and unseen cache sizes.
- Learned policies (ridge, trees, GBM, GELU MLPs, LinUCB, Thompson sampling) did not beat the rule and transferred worse.
- Cache state was the only decisive runtime signal; utilization, memory and history signals hurt learned policies.

What is not known. All of this was measured on one RTX 4070 with a PCIe gen4 x4 link and one model (gpt-oss-20b, 32 experts per layer, top_k 4). The rule is written in ratios (rho, top_k / E, residency in tokens) so that it can transfer, but whether it does is untested. Different GPUs change B_P, VRAM and cache size; different models change E, top_k, expert size and routing locality.

2. Research questions

#	Question	What would answer it
Q1	Does the reuse-aware rule beat q* on every (GPU, model) cell?	Paired end-to-end speedup with a 95% interval above 1 in each cell
Q2	Does q* become adequate when PCIe is fast (high B_P / B_H)?	q* vs best policy as rho rises from the 4070 to the 5090
Q3	Is the rule's fetch cost model right on other hardware?	Measured layer-time table wall(m, q) per GPU vs the rule's delta(m, q)
Q4	Does p = top_k / E transfer across models with different E and top_k?	Constant prior vs measured recent frequency, per model
Q5	Do the important runtime variables stay the same?	Feature importance and group ablation repeated per cell
Q6	Does a model trained on some cells predict unseen ones?	Leave-one-architecture-out and leave-one-model-out evaluation
Q7	What is the smallest feature set that generalizes?	The smallest set whose policy is never worse than q* on any cell

Table 1. Questions the cross-architecture data must answer.

3. What is collected: episodes

The unit of data collection is the **episode**, exactly as in the completed study, so new data is directly comparable with the existing 402,336 logged decisions.

Step	What happens
1	The server is already running; three burn-in episodes per server are run and discarded.
2	A control file selects the policy for this episode; the GPU expert cache is reset (same cold start for every episode).
3	Warmup: one standardized 128-token generate with that policy.
4	Measured: one 128-token generate with that policy; the client records ms/token, p50/p95/p99 and output hash.
5	For every MoE layer of every token the hook logs one decision row: pre-action features, the action q, and the outcomes.

Table 2. One episode.

Each decision row contains three strictly separated parts:

- **Pre-action features** (computed before q is chosen): m, layer, decode position, cache occupancy, free slots, LRU victim age, insertions/evictions/churn, hit-rate history, recent routing frequency of the missing experts, previous CPU / PCIe / GEMM times, CPU and GPU utilization, free GPU and host memory, rolling history windows.
- **Action**: q actually executed (0 to m).
- **Post-action outcomes**: layer wall time, CPU task time, PCIe copy time and GEMM time (CUDA events), evictions, decision overhead. Future quantities are built later only as offline labels.

Each (GPU, model) cell collects:

Phase	Policies	Cache sizes	Repeats	Episodes (approx.)
Exploration	uniform-random q, epsilon-exploration around q = 0 / 25% / 50%, m-threshold tables, the five fixed splits	3	1	48
On-policy	reuse-aware rule and ridge with epsilon-exploration	3	1	18
Evaluation	q*, q = 0, 25%, 50%, 75%, m, reuse-aware rule (and variants), ridge, LinUCB, Thompson sampling	3	3	about 120
Generalization	q*, 25%, rule, ridge on a held-out prompt and held-out cache sizes	6	2	about 50

Table 3. Episodes per (GPU, model) cell. About 240 episodes; roughly 4 to 6 GPU-hours on gpt-oss-20b-class models.

4. Architecture and model matrix

4.1 GPU architectures

GPU	Architecture	VRAM	Host link (nominal)	Status
RTX 4070	Ada Lovelace	12 GB	PCIe 4.0; measured x4 on the reference host (B_P = 6.7 GB/s)	Reference - data exists
RTX 4090	Ada Lovelace	24 GB	PCIe 4.0 x16	Planned
RTX 5090	Blackwell	32 GB	PCIe 5.0 x16	Planned
Others (for example RTX 3090, L40S, A100, H100)	as available	-	-	Added when access is available

Table 4. GPU architectures. Nominal link speeds are vendor specifications; the actual link, B_P and B_H are measured per host at Stage 0.

These GPUs differ in exactly the quantities the rule depends on: a wider and faster PCIe link raises B_P and therefore rho = B_P / B_H, and more VRAM allows a larger expert cache. The host CPU sets B_H, so CPU model, core count and memory are recorded for every machine; two machines with the same GPU but different CPUs are separate cells.

4.2 MoE expert models

Profile	Checkpoint (pinned revision)	MoE layers	Experts / layer	top_k	top_k / E	Expert format
gpt_oss_mxfp4	openai/gpt-oss-20b	24	32	4	0.125	MXFP4, 13.2 MB per expert
qwen_nvfp4	nvidia/Qwen3.6-35B-A3B-NVFP4	40	256	8	0.031	NVFP4, 1.8 MB per expert
gemma_nvfp4	nvidia/Gemma-4-26B-A4B-NVFP4	30	128	8	0.063	NVFP4

Table 5. Expert models (pins in configs/pruning/models.json). They span a 4x range of top_k / E and a 7x range of expert size.

4.3 Cell matrix

	gpt-oss-20b	Qwen3.6-35B-A3B	Gemma-4-26B-A4B
RTX 4070	done (reference)	planned, if host RAM holds the expert bank	planned
RTX 4090	planned	planned	planned
RTX 5090	planned	planned	planned
others	planned	planned	planned

Table 6. Target cells. A cell is attempted only after it passes Stage 0 qualification (Section 5).

Cache sizes are set relative to each model, not as fixed slot counts: three rates of the (layer x expert) pool chosen to give low, medium and high miss pressure, each at least the engine's minimum (2 x experts per layer when prefill overlap is on) and within the GPU's VRAM. The chosen rates are recorded in each cell's plan file.

5. Protocol on each machine

Stage 0 - setup and qualification

```
git clone git@github.com:ankitsblade/FreeToken-Reproduce.git
cd FreeToken-Reproduce
bash scripts/setup.sh --analysis-only
bash scripts/doctor.sh --out results/machine_before_runtime.json
bash scripts/setup.sh                       # installs and compiles the pinned FreeToken runtime
bash scripts/doctor.sh --require-runtime --out results/machine_after_runtime.json
source scripts/env.sh                       # under bash
```

- Download the pinned checkpoint for each model (Section 3 of docs/REPRODUCING.md).
- Run the bandwidth calibration on that machine to measure B_P and B_H for each expert format.

- Run the unpruned baseline for each model: `bash scripts/baseline.sh --profile gpt_oss_mxfp4 --model $MODEL --gpu 0 --repeats 3`.
- Run the runtime-q pilot plan and confirm that the hook reproduces the engine's q* (identical decisions and matching throughput).
- A cell that fails here - missing kernels for a quantization format on that GPU, insufficient VRAM or host RAM, loader errors - is recorded as a failure with its logs and is not collected.

Stage 1 - episode collection

```
bash scripts/run.sh runtime_q run --plan configs/runtime_q/collect_plan.json --model $MODEL --gpu 0
bash scripts/run.sh runtime_q run --plan configs/runtime_q/round2_plan.json  --model $MODEL --gpu 0
bash scripts/run.sh runtime_q run --plan configs/runtime_q/main_plan.json    --model $MODEL --gpu 0
bash scripts/run.sh runtime_q run --plan configs/runtime_q/generalization_plan.json --model $MODEL
--gpu 0
```

Every invocation creates `results/runs/runtime_q/UTC_UUID/` with `run.json` (exact command, times, status), `environment.json` (CPU, RAM, GPU, driver, package versions, Git state, engine hashes), console and server logs, and the episode and decision files. The driver skips episodes already present, so an interrupted run resumes.

Stage 2 - transfer of data

Run directories are large and are not committed to Git (`results/` is ignored). Each machine archives its run directories, verifies the archive, and transfers it to the shared store; the analysis machine imports them into a fresh study root. Raw run directories are never edited after collection.

Stage 3 - analysis

- Per cell: the same tables as the single-machine study - policy comparison with paired 95% intervals, single-step cost table wall(m, q), per-fetch reuse analysis, feature importance and group ablations.
- Across cells: pool the decision data with machine and model identifiers; evaluate leave-one-architecture-out and leave-one-model-out; compare the rule's predicted fetch cost with each GPU's measured cost table.
- Prefer dimensionless quantities (rho, top_k / E, residency in tokens, miss ratio) when fitting anything across cells.

6. Controls that keep cells comparable

Control	Rule
Held constant across all cells	batch 1, greedy decode, 128 measured tokens, one warmup generate, eager decode, naive KV cache, the same episode protocol and policies
Prompts	Frozen token-ID prompts. The GPT-OSS chat template embeds the local date, which changes the prompt and the routing, so rendered chat text must not be re-generated on each machine.
One GPU job at a time	Concurrent servers corrupt timings. No heavy CPU work (for example checkpoint-scale tensor jobs) beside a GPU run.
Burn-in and order	Three discarded burn-in episodes per server; policy order rotated across repeats.
Memory	Runs execute in a scope without swap; RAM or swap pressure rejects the whole paired unit.
Separate arms	A change of quantization, cache size, driver, runtime source or execution mode is a separate arm, not a repeat.
Correctness	Output hashes recorded per episode. Divergent text between policies is documented, since CPU and GPU expert kernels are not bit-identical.

Table 7. Measurement hygiene, carried over from the single-machine study.

7. Why the GitHub repository makes this possible

The repository **ankitsblade/FreeToken-Reproduce** packages everything a new machine needs, so the collection is identical everywhere:

- **Pinned runtime.** FreeToken is restored from a recorded base commit plus a versioned experiment-hook patch and compiled on the destination machine; compiled binaries are never copied between machines.
- **Pinned models.** Checkpoint repositories and revisions are recorded; the baseline checks model geometry against them.
- **Environment capture.** `scripts/doctor.sh` and each run's `environment.json` record the exact hardware and software.
- **One entry point.** `scripts/run.sh` runs any study package with recorded commands; the runtime-q plans live in `configs/runtime_q`.
- **Attribution.** Each study package keeps its original contributor (runtime-q and split policy: Saketh; prefill overlap: Kushal; routing locality: Yeshwanth; bandwidth and pruning: Ankit).

Scope of 'any architecture'. The supported target is Linux x86_64 with an NVIDIA GPU and a driver compatible with the pinned CUDA 13 runtime. Each new GPU must still pass Stage 0: kernels for a given quantization format can be missing or behave differently on a new architecture, and that is recorded as a failure rather than worked around.

8. Deliverables and success criteria

- A pooled, versioned decision dataset with machine and model identifiers and the environment record of every run.
- Per-cell result tables and figures in the same format as the single-machine report.
- A cross-cell report answering Q1 to Q7, stating exactly which GPUs and models were measured.
- A final q-selection function: the smallest feature set and simplest form that is never worse than q* on any measured cell and is better on most. If q* is adequate on fast-PCIe machines, that is reported as a result.

The rule is considered to **transfer** if, in every qualified cell, its paired speedup over q* has a 95% interval at or above 1, without re-tuning its constant. Cells where it fails are reported with the measured cost table that explains why.

9. Risks

Risk	Mitigation
NVFP4 / MXFP4 kernels unavailable or different on a GPU	Stage 0 qualification; record and skip the cell
Host RAM too small for a model's expert bank	Skip that (machine, model) cell; record the requirement
Eager-mode overhead differs across GPUs	Compare policies only within a cell; also record the CUDA-graph q* baseline per cell
Very fast PCIe makes every policy fetch everything	Expected and informative (Q2); keep q* and q = m as anchors
Different CPUs on the same GPU model	Treat as separate cells; B_H is measured per host
Limited access time on borrowed machines	Run Stage 0 and the evaluation phase first; exploration later if time allows

Table 8. Risks and mitigations.